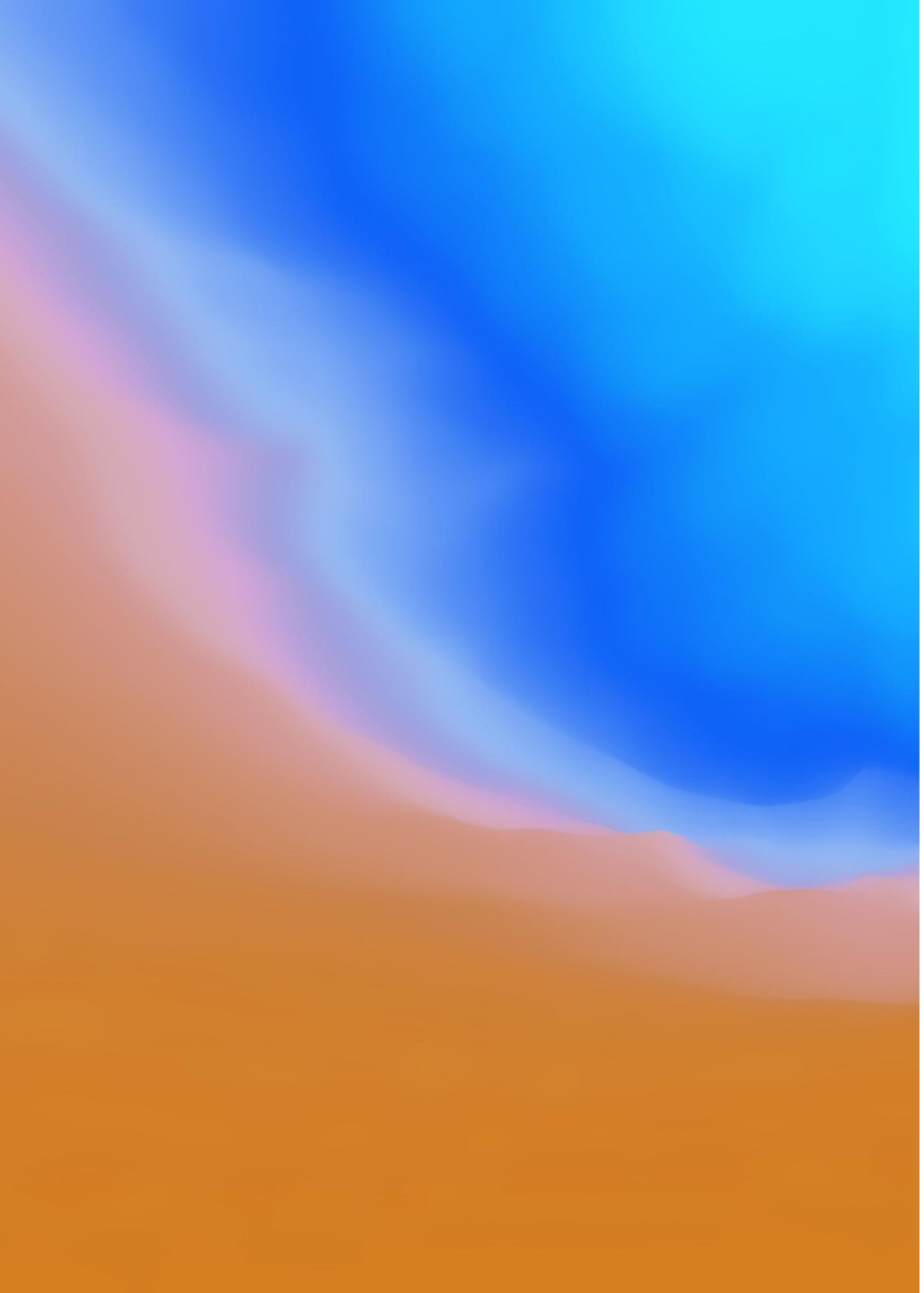
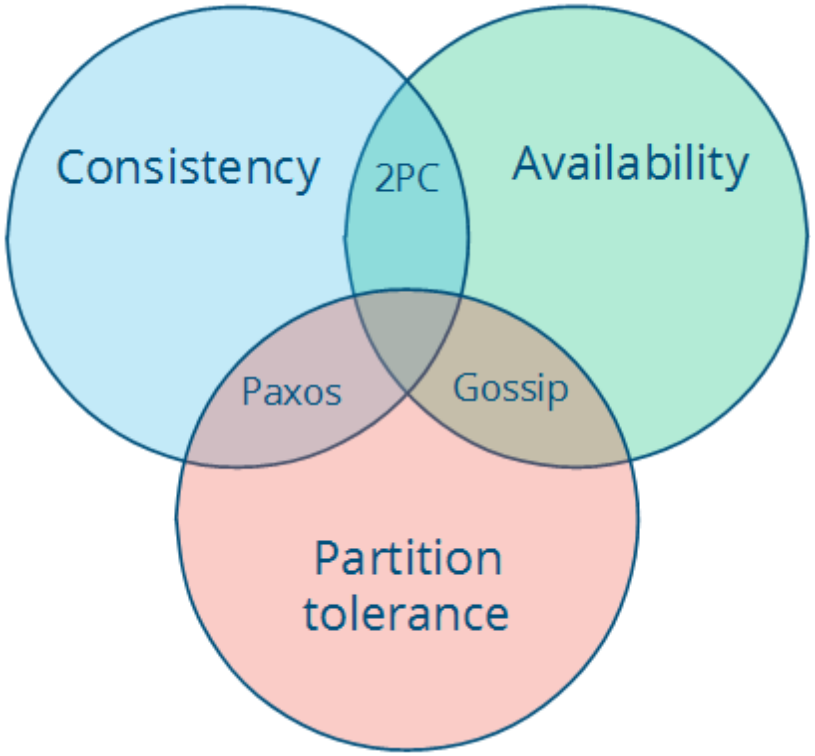


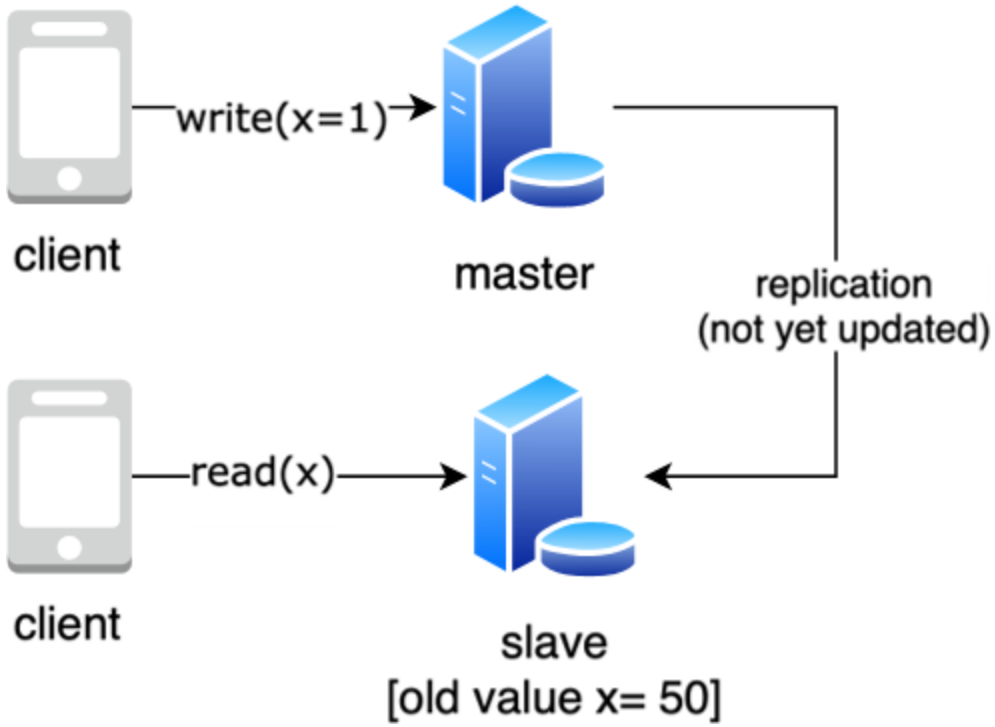
MIDTERM MASTERY

WHAT I HAVE ENJOYED LEARNING SO FAR

- CAP Theorem Explains Master-Slave Read Inconsistency
- MapReduce: From Paper to Practice
- Auto-Scaling in Product-API

CAP THEOREM EXPLAINS MASTER-SLAVE INCONSISTENCY

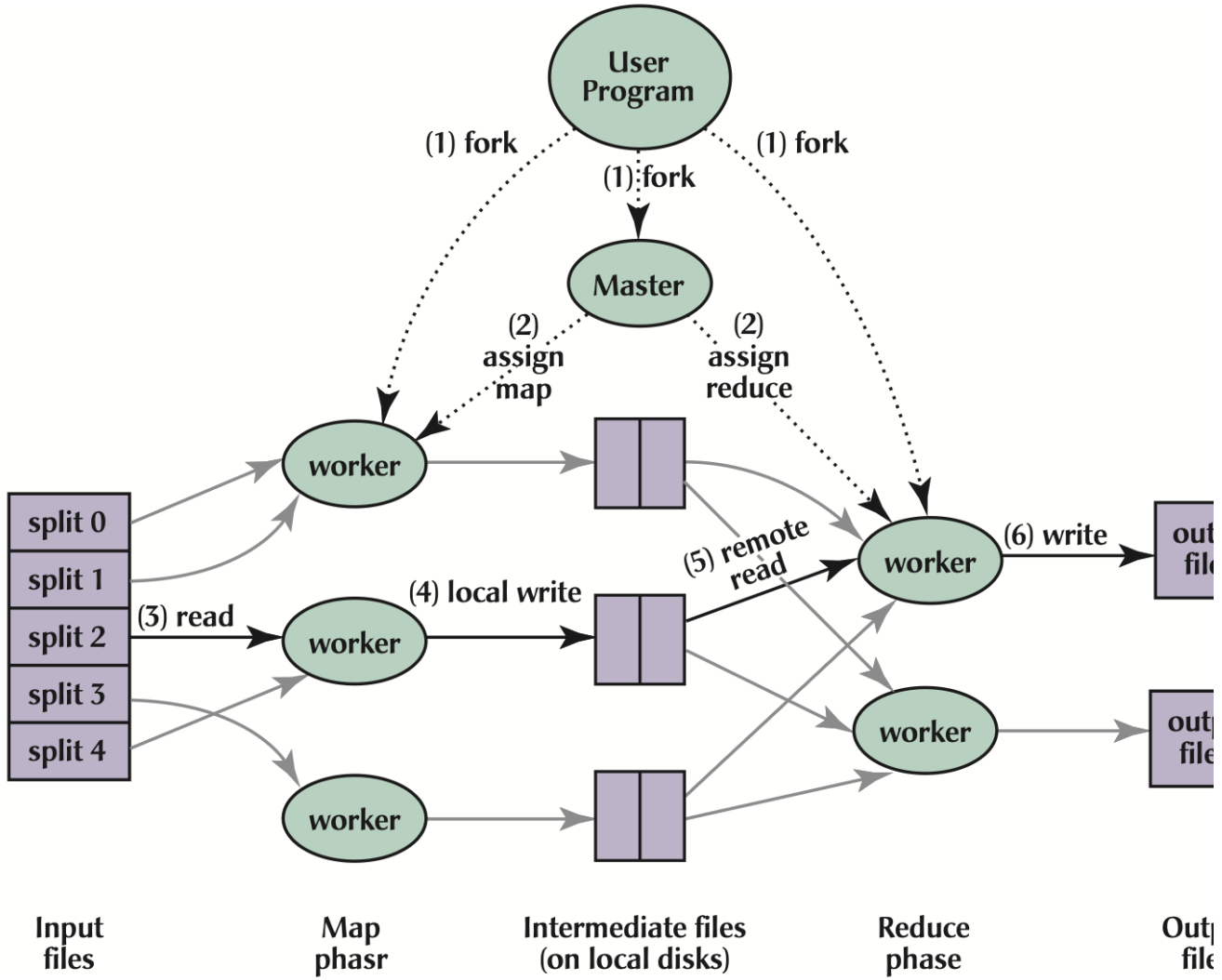




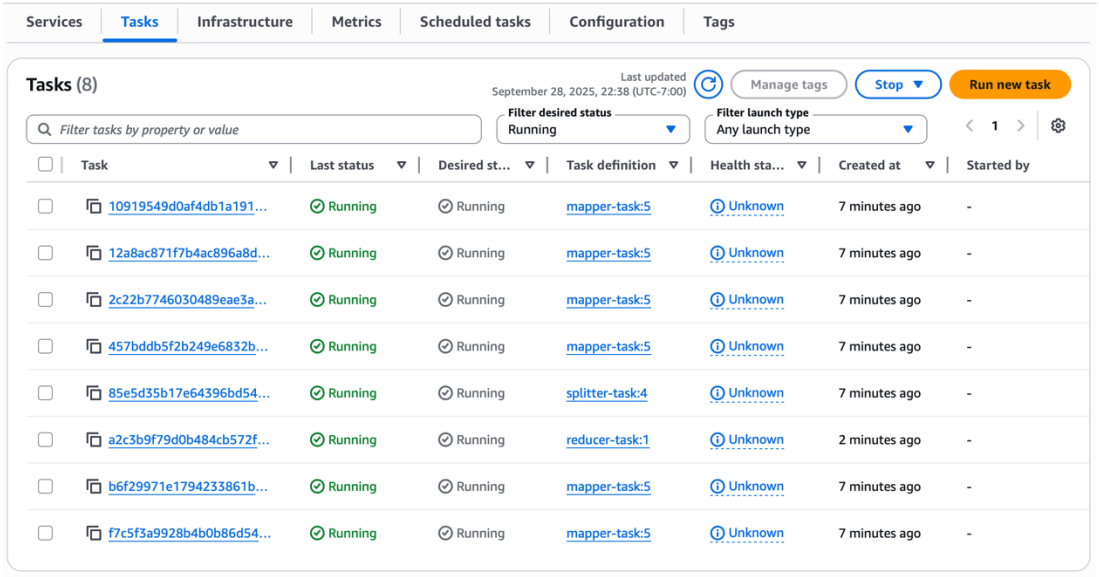
MAPREDUCE: FROM PAPER TO PRACTICE

MapReduce = Simple Abstraction + Powerful Framework

- Map: Transform data in parallel
- Reduce: Aggregate results
- Framework: Handles failures, scheduling, distribution

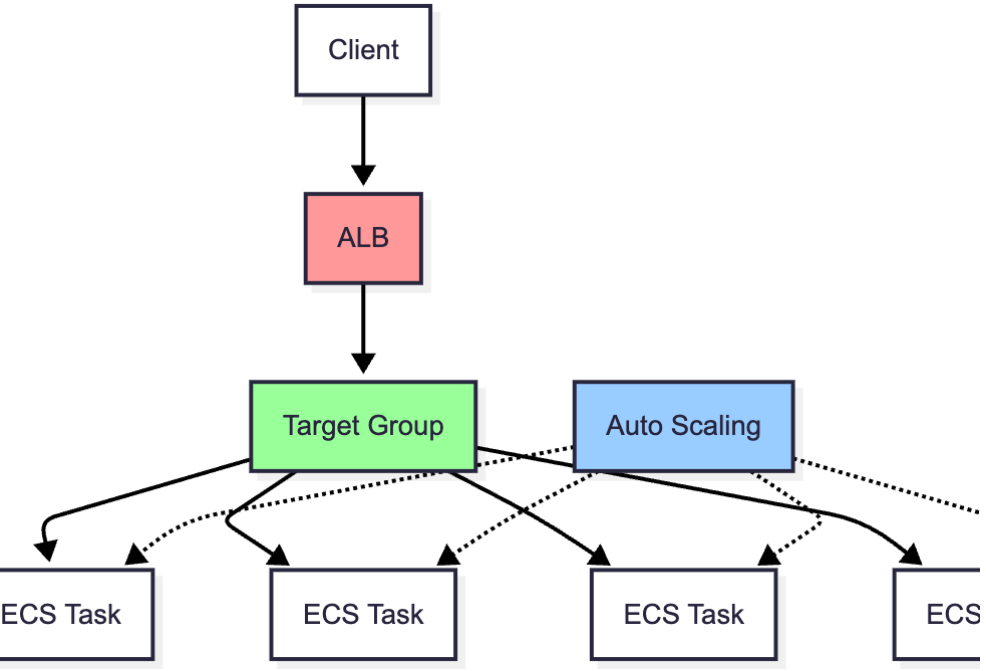


MAPREDUCE: FROM PAPER TO PRACTICE




```python
### Perform Map Reduce Steps ###
# 1. Split the input file
split_result = await self.http_get_json(
    f"http://{splitter_ip}:8080/split",
    params={"url": input_url, "chunks": str(chunks)}
)

chunk_urls = split_result.get("chunk_urls", [])

# 2. Map each chunk
map_tasks = []
for i, chunk_url in enumerate(chunk_urls):
    mapper_ip = mapper_ips[i % len(mapper_ips)]
    map_tasks.append(self.http_get_json(
        f"http://{mapper_ip}:8080/map",
        params={"url": chunk_url}
    ))

map_results = await asyncio.gather(*map_tasks)
print(f"Completed {len(map_results)} map operations")

# 3. Reduce the mapped results
reduce_result = await self.http_get_json(
    f"http://{reducer_ip}:8080/reduce"
)
```

AUTO-SCALING IMPLEMENTATION

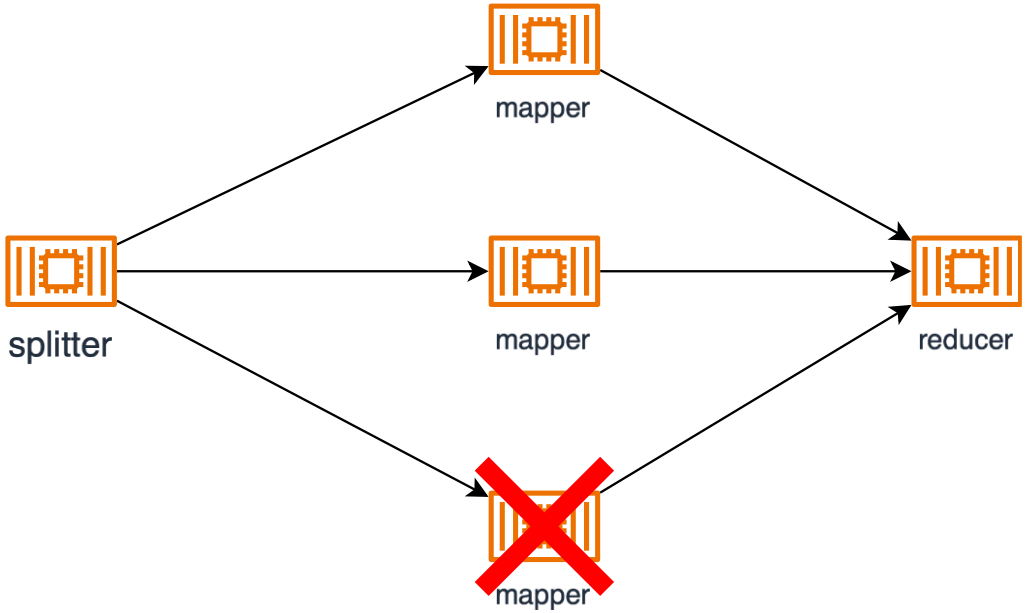


Users	Task Count	CPU Usage (%)	Memory Usage (%)	RPS	50% Response Time (ms)	95% Response Time (ms)
0	2	3	6	N/A	N/A	N/A
20	2	40	10	9	40	200
40	2	70	14	17	70	440
60	3	80	15	25	55	430
80	4	75	18	34	47	340

CRASHING & RECOVERING

MapReduce Fault Tolerence

PARTIAL MAPPER FAILED




```
Downloading text from: s3://mapreduce-experiment-975050147762/50MB-TXT-FILE.txt
Downloaded 52433149 characters
Word counting completed in 1.84 seconds

Word Count Results:
Total words: 6883544
Unique words: 311
```

```
%2Fchunk_6.txt
Completed 6 map operations
HTTP GET: http://54.213.113.197:8080/reduce
MapReduce completed in 6.73 seconds
MapReduce Result: Successfully reduced all mapper results. Total: 5735753 words, Unique: 311 word
s
Total Words: 5735753
Unique Words: 311
(venv) wenshuangzhou@wenshuangs-MacBook-Pro CS6650 %
```

FAULT REASON

```
    # 2. Map each chunk
    map_tasks = []
    for i, chunk_url in enumerate(chunk_urls):
        mapper_ip = mapper_ips[i % len(mapper_ips)]
        map_tasks.append(self.http_get_json(
            f"http://{mapper_ip}:8080/map",
            params={"url": chunk_url}
        ))
          You, 3 weeks ago • add experiments
    map_results = await asyncio.gather(*map_tasks)
    print(f"Completed {len(map_results)} map operations")

    # 3. Reduce the mapped results
    reduce_result = await self.http_get_json(
        f"http://{reducer_ip}:8080/reduce"
    )
```

HOW TO FIX

```
# 2. Map each chunk
map_tasks = []
for i, chunk_url in enumerate(chunk_urls):
    mapper_ip = mapper_ips[i % len(mapper_ips)]
    map_tasks.append(self.http_get_json(
        f"http://{mapper_ip}:8080/map",
        params={"url": chunk_url}
    ))      You, 9 hours ago • add midterm part2

map_results = await asyncio.gather(*map_tasks)

for i, result in enumerate(map_results):
    if isinstance(result, dict) and "error" in result:
        alternate_mapper_ip = self._get_alternate_mapper(i)
        if alternate_mapper_ip:
            chunk_url = chunk_urls[i]
            alternative_result = await self.http_get_json(
                f"http://{alternate_mapper_ip}:8080/map",
                params={"url": chunk_url}
```

```
Completed 6 map operations
HTTP GET: http://54.213.113.197:8080/reduce
MapReduce completed in 9.21 seconds
MapReduce Result: Successfully reduced all mapper results. Total: 6883544 words, Unique: 311 words
Total Words: 6883544
Unique Words: 311
```

```
mapper availab

print(f"Completed {len(map_results)} map operations")

# 3. Reduce the mapped results
reduce_result = await self.http_get_json(
    f"http://{reducer_ip}:8080/reduce"
)
```